

System Design Document (SSD)

TapNOrder – Scan. Order. Relax.

1. Overview

TapNOrder is designed to address a very common inefficiency in dine-in restaurants the dependency on waiters for even basic interactions like placing an order or checking its status.

In most cases, the process is slow, fragmented, and sometimes inaccurate due to miscommunication. Instead of trying to optimize that system, TapNOrder shifts the control directly to the customer.

The system allows users to:

- Scan a QR code placed on the table
- Access a digital menu instantly
- Place orders without waiting
- Track their order in real time

What looks like a simple interface is actually a coordinated system that connects the customer, backend logic, and restaurant operations in a continuous flow.



CCD VEG

Pick a mood



track order

pre book

burger

pizza

pasta

sandwich

wrap

Top items

recommended

people's choice

hot arrivals



Aloo tikki burger
rs 120

add to cart



Chicken cheese burger
rs 140

add to cart



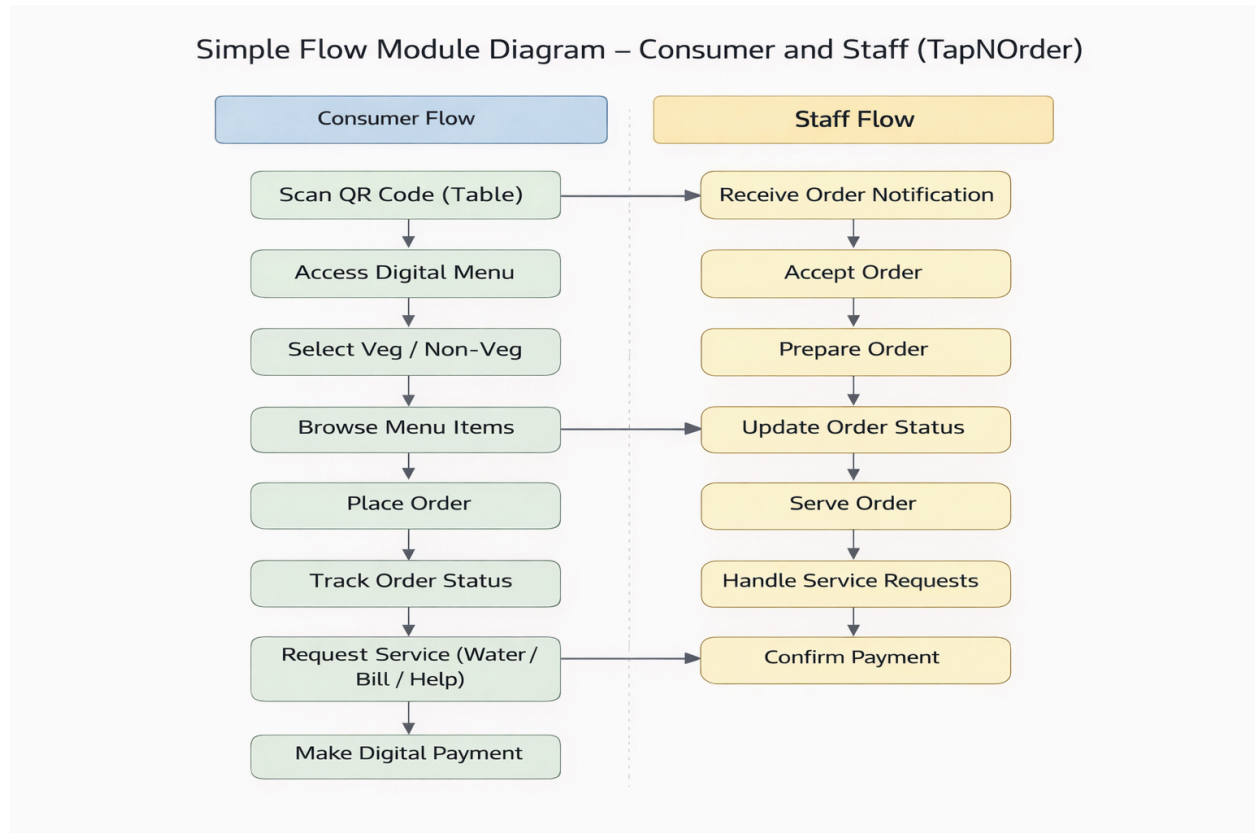
double tikki burger
rs 170

add to cart



2. System Architecture

The system follows a three-layer architecture, but more importantly, each layer has a clearly defined responsibility. The goal is to avoid unnecessary overlap and keep the system modular.



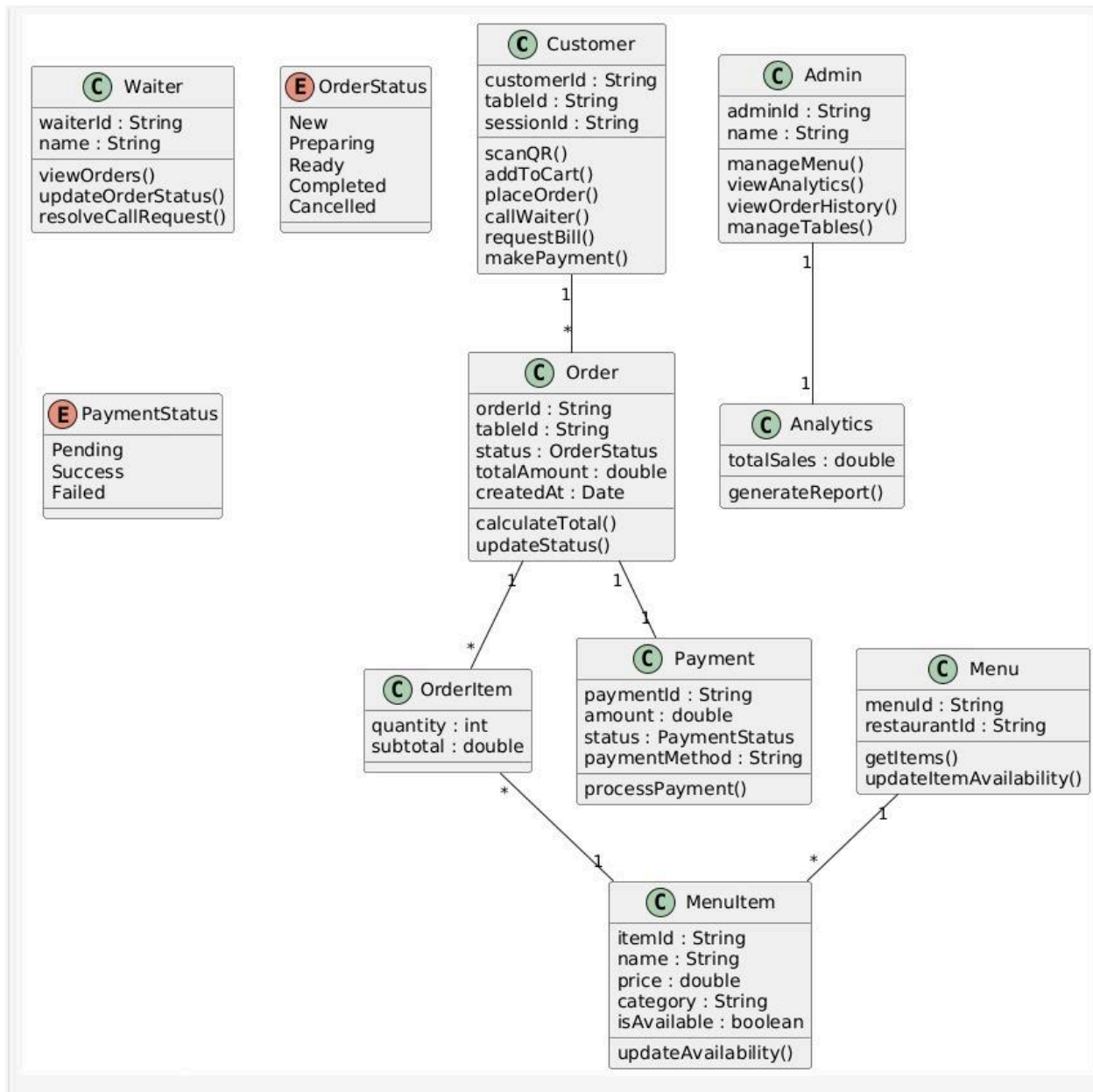
Frontend Layer (Client-Side Interaction)

This is the interface that both customers and restaurant staff interact with.

It includes:

- Menu browsing interface
- Cart system
- Order tracking screen
- Admin dashboard

The design priority here is speed and clarity. In a restaurant setting, users are not exploring features they want to complete tasks quickly. So the interface is intentionally minimal and action-driven.



Backend Layer (Application Logic)

This layer acts as the core processing unit of the system.

It handles:

- Order validation and processing
- User authentication
- Payment handling

- Status updates

Every interaction from the frontend is routed through the backend, processed, and then returned as a response. This ensures consistency and control over the system behavior.

Database Layer (Data Management)

This layer stores all persistent data required by the system.

It includes:

- User data
- Menu data
- Order details
- Payment records

The database is structured in a way that separates concerns across entities, which helps in maintaining clarity and scalability as the system grows.

3. Core Design Approach

Instead of just building features, the system is designed around three key principles.

1. Reducing Dependency

The system minimizes reliance on waitstaff for routine tasks. Customers can independently browse, order, and track without needing assistance.

2. Real-Time Visibility

At any point, the user should know what is happening with their order. This reduces uncertainty and improves overall experience.

3. Low Cognitive Load

The interface avoids unnecessary complexity. Actions are direct, labels are clear, and navigation is minimal. The system assumes that users want speed, not exploration.

4. System Modules

4.1 Customer Module

This module handles all user-facing interactions.

It includes:

- QR-based entry into the system
- Menu browsing with categorized items
- Cart management
- Order placement
- Order tracking

The design avoids forcing users into long processes. For example, ordering does not require multiple confirmation layers unless necessary.



Cart

[DONE](#)



Pizza Calzone
European



Rs 96

14"



1



TABLE NUMBER 1

[EDIT](#)

TOTAL: **Rs96**

[Breakdown >](#)

PLACE ORDER

4.2 Order Management Module

This is the operational core of the system.

It is responsible for:

- Receiving order requests
- Validating items and availability
- Storing order data
- Updating order status
- Synchronizing with the admin side

This module ensures that there is no breakdown between what the user requests and what the restaurant processes.

Add Image Here

Order tracking interface

Caption: Output of order management module from the user's perspective

4.3 Payment Module

This module handles transaction processing.

Supported methods include:

- Cash
- Card payments
- Online payment systems

The system is intentionally flexible to accommodate different user preferences, especially in contexts where digital payments are not always preferred.



Payment



Cash

VISA

Visa



Mastercard



Payl

No master card added

You can add a mastercard and
save it for later

pay by cash

pay at counter

+ ADD NEW

TOTAL: **Rs96**

PAY & CONFIRM

4.4 Admin Module

This module is used by restaurant staff to manage operations.

It includes:

- Viewing incoming and ongoing orders
- Updating order status
- Managing menu items
- Monitoring basic analytics

The goal here is not to replace staff but to provide them with a structured system that reduces manual coordination.



CCD
Near jkl



20

RUNNING ORDERS

05

ORDER REQUEST

Total Revenue
rs 2,241

Daily ▾

[See Details](#)



Reviews

[See All Reviews](#)

★ 4.9 Total 20 Reviews

Populer Items This Weeks

[See All](#)



5. Data Flow

The system operates through a continuous exchange of data between components.

A typical flow is as follows:

1. User scans QR code
2. Frontend sends request to backend
3. Backend retrieves menu data from database
4. User places an order
5. Order is stored in database
6. Admin interface reflects the new order
7. Status updates are sent back to the user

This is not a one-time flow — it continues as the order progresses.

6. Database Design

The database is structured around key entities, each with a specific role.

User

Stores user identity and login credentials.

Menu Item

Stores item details such as name, price, and category.

Order

Links users with selected items and tracks order status.

Payment

Stores transaction-related details.

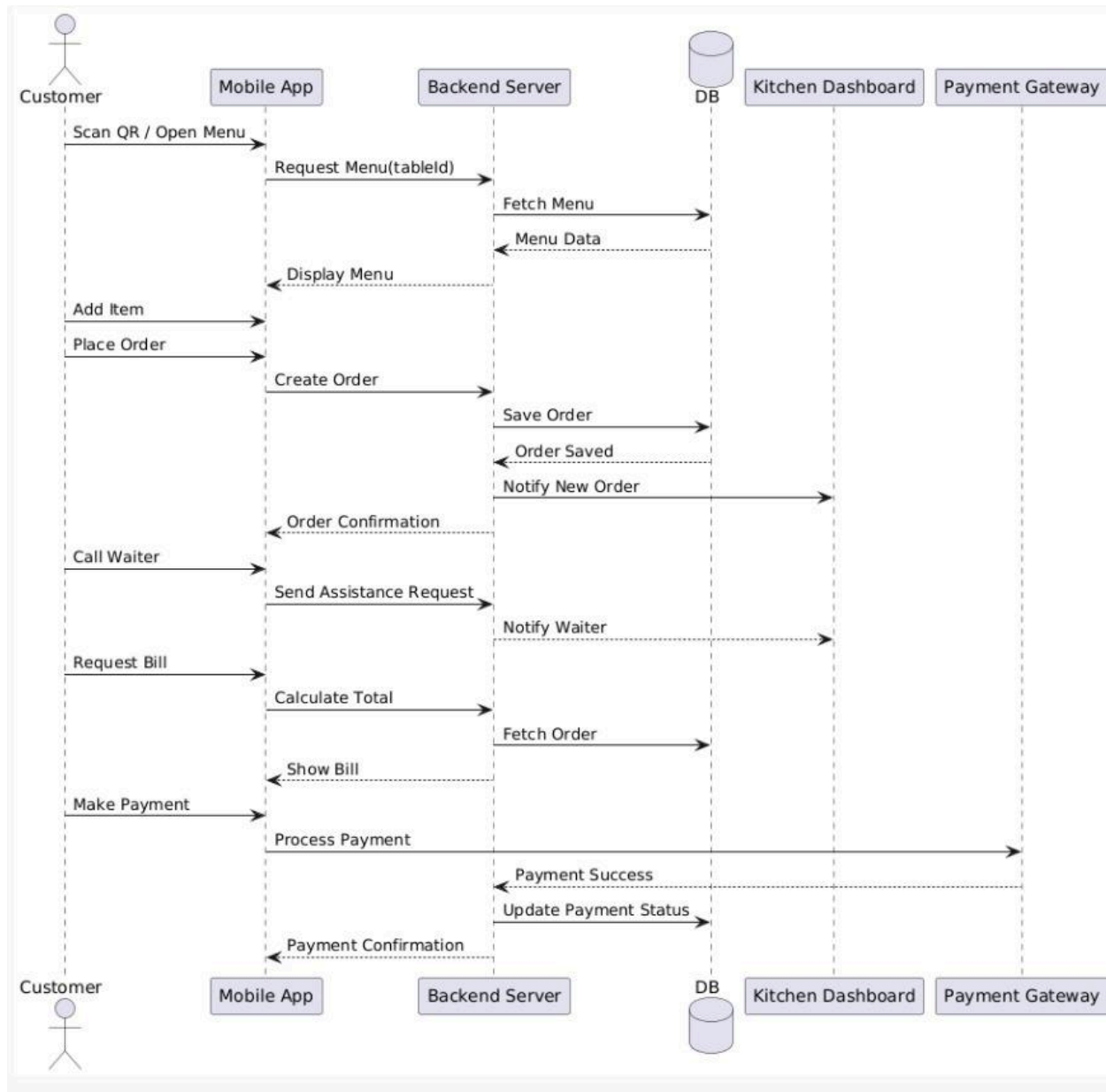
The separation of entities ensures that updates in one part of the system do not create inconsistencies elsewhere.

7. Sequence Flow (Order Placement Example)

To understand how the system behaves internally, consider the order placement process:

1. User selects items and confirms order
2. Request is sent to backend
3. Backend validates the request
4. Order is stored in database
5. Admin module is updated
6. Confirmation is sent to the user

This sequence ensures that every action is verified before being finalized.



8. Scalability and Performance Considerations

The system is designed to handle:

- Multiple users placing orders simultaneously
- Multiple restaurants using the system
- Real-time updates without delay

The modular architecture allows components to be scaled independently if needed.

9. Constraints

- Requires stable internet connectivity
 - Depends on third-party payment services
 - Requires a smartphone for access
-

10. Future Enhancements

The system can be extended with:

- Recommendation systems based on user behavior
 - Voice-based ordering
 - Loyalty and reward programs
 - Multi-language support
-

Final Note

This system is not just about digitizing menus. It restructures how ordering works by reducing friction, improving visibility, and organizing backend operations in a more reliable way.